Comparativa entre la librería ML .NET (Machine Learning) y MathNet.Numerics (Estadística) en ajuste de curvas e interpolación en una serie de datos tipo Y = F(X)

Autor: Rafael Alberto Moreno Parra

Correo: Rafael-morenop@unilibre.edu.co

Abstract

This paper presents a comparative analysis of two libraries, ML .NET and MathNet.Numerics, in the context of curve fitting and interpolation for datasets of the form Y=F(X). The study aims to evaluate the accuracy and interpolation of both libraries through a series of experiments on synthetic data. Key metrics such as mean squared error (MSE), computational efficiency, and ease of implementation are used to assess the strengths and weaknesses of each library.

Resumen

Este artículo presenta un análisis comparativo de dos librerías, ML .NET y MathNet.Numerics, en el contexto del ajuste de curvas e interpolación para conjuntos de datos de la forma Y=F(X). El estudio tiene como objetivo evaluar la precisión y la interpolación de ambas librerías a través de una serie de experimentos con datos sintéticos. Se utilizan métricas clave como el error cuadrático medio (MSE), la eficiencia computacional y la facilidad de implementación para evaluar las fortalezas y debilidades de cada librería.

Datos sintéticos

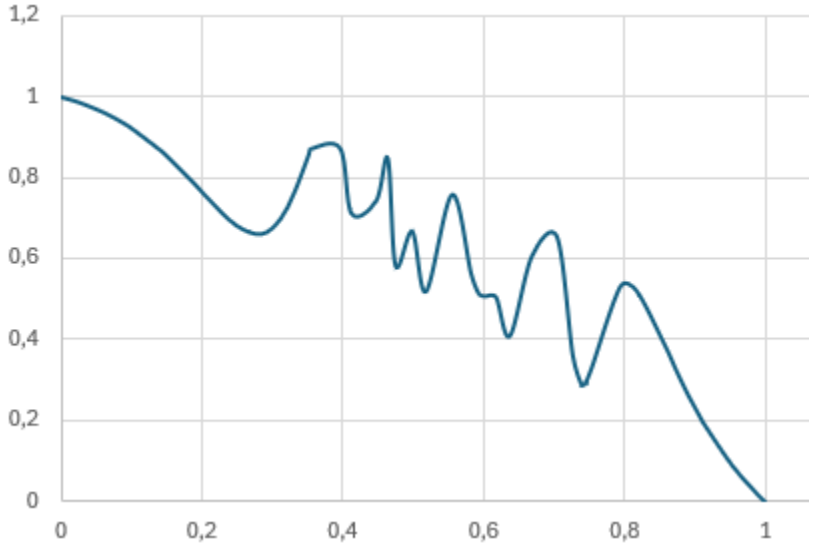
Se busca un patrón en una serie de datos XY. Significa que no hay comportamiento aleatorio, luego esa serie nace de una ecuación determinada $Y = f(X)$

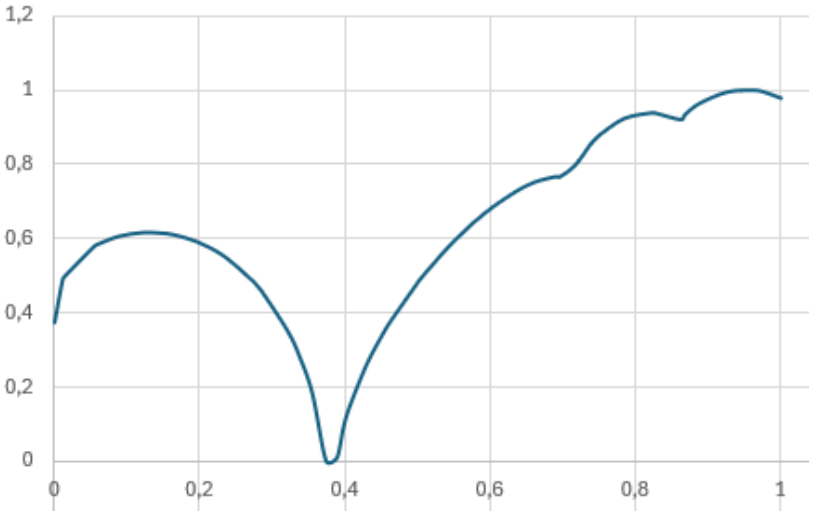
Para probar ambas librerías (ML .NET y MathNet.Numerics) se crea en forma aleatoria una ecuación Y = F(X), y con esta ecuación se generan los datos XY. Los beneficios son:

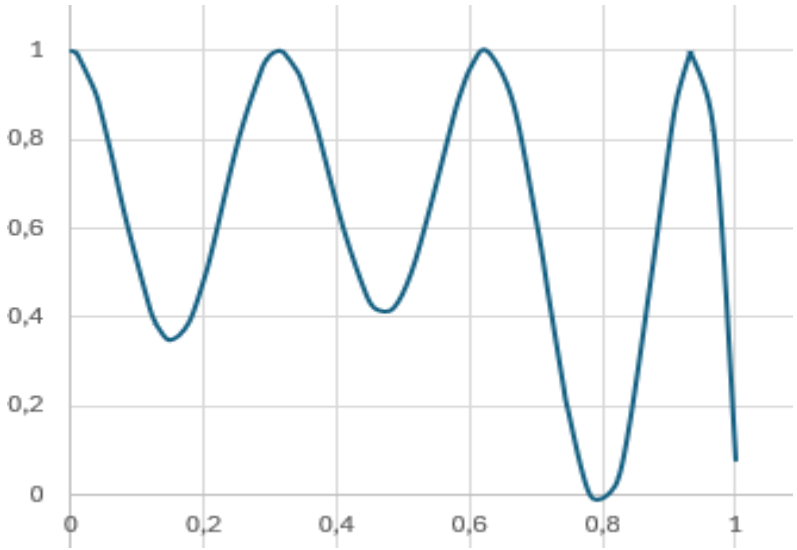
1. Hay datos, de cualquier número de registros de tipo numérico.
2. Se garantiza una relación causal porque Y es la variable dependiente y X es la variable independiente (la causa).
3. Se puede generar cualquier ecuación.
4. Los datos se generan en memoria, no es requerido hacer operaciones de entrada-salida.

Para generar la ecuación en forma aleatoria se hace uso de la estructura de datos: árbol binario balanceado. En el nodo raíz hay una operación matemática (+, -, *, /) y una función (seno, coseno, tangente, valor absoluto, raíz cuadrada) y en las ramas o es la variable X o es una constante numérica. Con esta estructura se puede generar cualquier ecuación que incluye el uso de paréntesis.

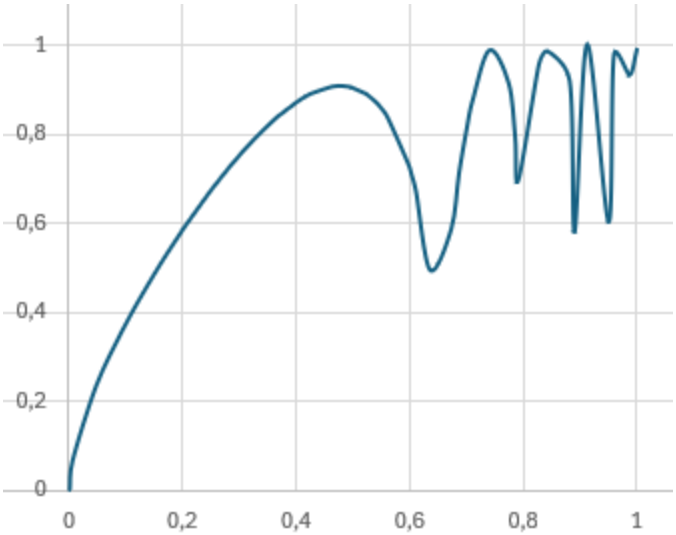
Ejemplos de datos generados:







Autor: Rafael Alberto Moreno Parra



El problema del sobre-entrenamiento

Es posible encontrar una curva que acierte en cada punto XY y es el polinomio. Si hay N valores XY, habrá un polinomio de grado N-1 que pase por cada punto. Sin embargo, esta curva tiene el problema que no es útil para hacer predicciones (no sirve para interpolar, ni para extrapolar). A ese problema se le llama sobre-entrenamiento. En búsqueda de la mejor curva útil, es necesario que una vez encontrada una curva, hay que evaluarla con puntos XY que no hayan sido usados para el entrenamiento. Luego se deduce que la mejor curva es aquella que es más precisa a ese nuevo conjunto de puntos.

ML .NET

Es una biblioteca de código abierto desarrollada por Microsoft que permite a los desarrolladores .NET integrar capacidades de aprendizaje automático (ML) en sus aplicaciones. Características:

- Compatibilidad con modelos preentrenados: Utiliza modelos de TensorFlow y ONNX.
- Interoperabilidad con entornos de datos: Soporta formatos como CSV, SQL, entre otros.
- Facilidad de uso: Proporciona una API fácil para desarrolladores .NET.
- Capacidades de aprendizaje supervisado y no supervisado: Incluye soporte para regresión, clasificación, clustering y detección de anomalías.

ML .NET ha ganado tracción en la comunidad de desarrolladores .NET debido a su integración con Visual Studio y su facilidad de uso. Según GitHub, el repositorio de ML .NET tiene más de 3,000 estrellas. Con 8.655.340 de descargas a 09 de febrero de 2025 en NuGet. Licencia MIT.

Preparación de la librería:

```
var ContextoML = new MLContext();
var Datos = ContextoML.Data.LoadFromEnumerable(DatosEvalua);

// Dividir los datos en entrenamiento y testeo
var PreparaDatos = ContextoML.Data.TrainTestSplit(Datos, testFraction: 0.2);
var DatosEntrenamiento = PreparaDatos.TrainSet;
var DatosTesteo = PreparaDatos.TestSet;
```

Preparación de FastTree (un añadido de ML .NET):

```
var Aprendizaje = ContextoML.Transforms.Concatenate("Caracteristicas", "X")
                .Append(ContextoML.Transforms.NormalizeMinMax("Caracteristicas"))
                .Append(ContextoML.Regression.Trainers.FastTree(new
FastTreeRegressionTrainer.Options {
                    NumberOfLeaves = 50, // Ajusta según sea necesario
                    MinimumExampleCountPerLeaf = 5, // Ajusta según sea necesario
                    LearningRate = 0.1, // Ajusta según sea necesario
                    NumberOfTrees = 200, // Ajusta según sea necesario
                    LabelColumnName = "Y",
                    FeatureColumnName = "Caracteristicas"
                }));
```

Entrenamiento de ML .NET con FastTree:

```
// Entrenar el modelo
var ModeloEntrenado = Aprendizaje.Fit(DatosEntrenamiento);

// Hacer transformaciones
ModeloEntrenado.Transform(DatosTesteo);

// Crear un motor de predicción
var MotorPrediccion = ContextoML.Model.CreatePredictionEngine<Dato,
ModeloPrediccion>(ModeloEntrenado);
```

Prueba con los datos no usados para entrenar:

```
//Datos no usados en el entrenamiento
List<Dato> DatosProbar = [];
for (int registro = 0; registro < Program.Datos.Test.Count; registro++) {
    float Xval = (float)Program.Datos.Test[registro].ValorX;
    float Yval = (float)Program.Datos.Test[registro].ValorY;
    DatosProbar.Add(new Dato { X = Xval, Y = Yval });
}

//Compara con los datos de testeo
var DatosTest = ContextoML.Data.LoadFromEnumerable(DatosProbar);
var DatosEnum = ContextoML.Data.CreateEnumerable<Dato>(DatosTest, reuseRowObject:
false).ToList();

double Acumula = 0;
for (int i = 0; i < DatosEnum.Count; i++) {
    var Fila = DatosEnum[i];
    var predice = MotorPrediccion.Predict(new Dato { X = Fila.X });
    double Diferencia = Program.Datos.Test[i].ValorY - predice.Y;
    Acumula += Diferencia * Diferencia;
}
return Acumula;
```


Autor: Rafael Alberto Moreno Parra

MathNet.Numerics

Es una librería de código abierto que proporciona una extensa colección de algoritmos y tipos de datos para realizar cálculos numéricos en C#. Esta librería abarca áreas como álgebra lineal, estadísticas, optimización, interpolación y transformadas. Características:

- Estadísticas básicas y avanzadas: Métodos para calcular media, desviación estándar, percentiles, entre otros.
- Álgebra lineal: Soporte para matrices y vectores, incluyendo operaciones avanzadas.
- Interpolación y optimización: Herramientas para ajustar curvas y resolver problemas de optimización.

En GitHub, el repositorio de MathNet.Numerics tiene más de 2,000 estrellas. Con 41.273.264 de descargas a 09 de febrero de 2025 en NuGet. Licencia MIT.

El desarrollador debe escribir un modelo de ecuación y la librería se encarga de ajustar los coeficientes de esa ecuación para lograr la mejor precisión con los datos dados:

```
// Función modelo personalizada a*seno(b*x+c) + ..... + p*seno(q*x+r)
static double Modelo(double X, Vector<double> Parametros) {
  double Resultado = 0;
  for (int Cont = 0; Cont < Parametros.Count; Cont += 3)
    Resultado += Parametros[Cont] * Math.Sin(Parametros[Cont + 1] * X + Parametros[Cont + 2]);
  return Resultado;
}
```

Se hace el cálculo del error cuadrático medio (MSE):

```
// Función de error (suma de los cuadrados de los errores)
static double FuncionError(Vector<double> Parametros) {
  double Error = 0;
  for (int Cont = 0; Cont < Datos.Registros.Count; Cont++) {
    double Ypredice = Modelo(Datos.Registros[Cont].ValorX, Parametros);
    Error += (Datos.Registros[Cont].ValorY - Ypredice) * (Datos.Registros[Cont].ValorY -
Ypredice);
  }
  return Error;
}
```

Uso de la librería:

```
var Resultado = FindMinimum.OfFunction(
  function: FuncionError,
  initialGuess: ValorInicial,
  tolerance: 0.001, // Ajusta la tolerancia según sea necesario
  maxIterations: 6000 // Incrementa el número máximo de iteraciones
);
```

Prueba con los datos no usados para entrenar:

```
//Compara los valores Test con la curva hallada
double Acumula = 0;
for (int Cont = 0; Cont < Datos.Test.Count; Cont++) {
  double x = Datos.Test[Cont].ValorX;
  double y = Modelo(x, Resultado);
  double Diferencia = Datos.Test[Cont].ValorY - y;
  Acumula += Diferencia * Diferencia;
}
return Acumula;
```


Autor: Rafael Alberto Moreno Parra

Algoritmo para probar ambas librerías

Los pasos realizados para evaluar ambas librerías son los siguientes:

Paso 1: Generar una ecuación Y=F(X) en forma aleatoria.

Paso 2: Con esa ecuación se generan N puntos XY.

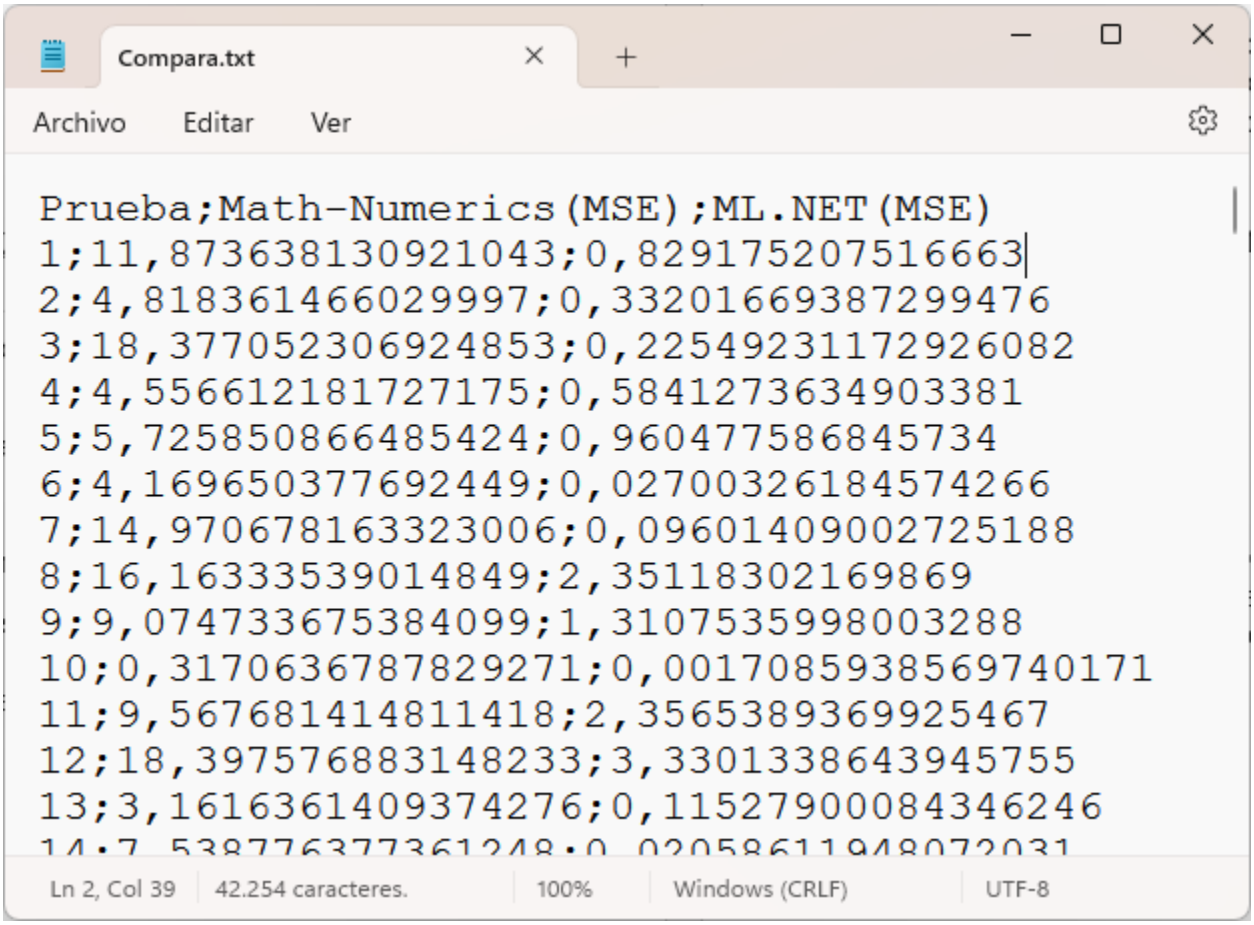
Paso 3: En forma aleatoria se toma el 10% de los N puntos XY y se almacena en un listado llamado TEST.

Paso 4: El 90% restante de los N puntos XY se usa para entrenar o buscar la curva con cada librería.

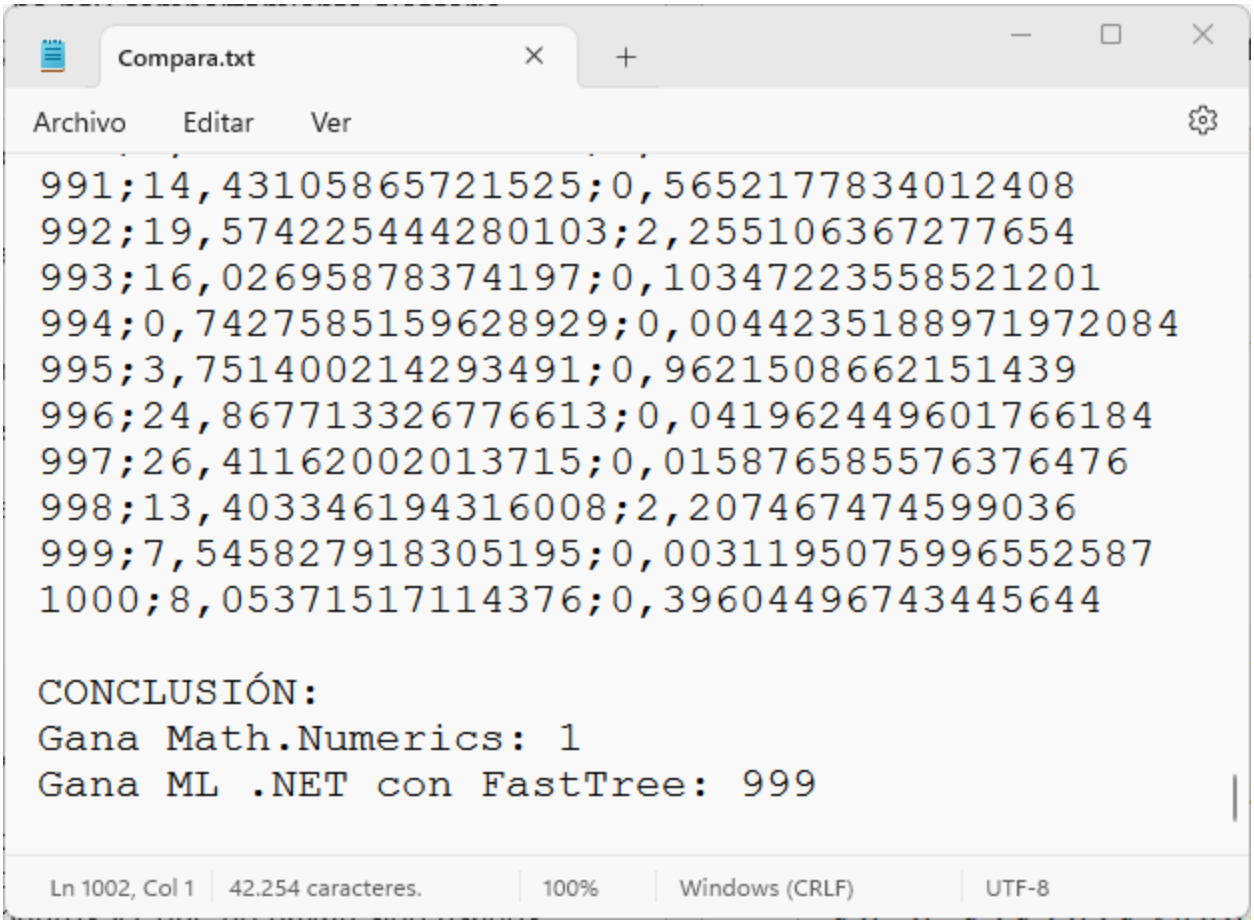
Paso 5: Cada librería (ML .NET y Math.Numerics) ofrece una curva que mejor se ajusta a los puntos XY dados.

Paso 6: Se toma la curva ofrecida por Math.Numerics y se evalúa su ajuste (con la métrica de error cuadrático medio MSE) con el listado TEST. Se hace el mismo procedimiento con la curva ofrecida por ML .NET. Se considera mejor, la curva que tenga el valor MSE más bajo.

Los pasos 1 al 6 se repitieron 1000 veces. Y cada vez se generaron 3000 puntos XY. Estos fueron los resultados:



Autor: Rafael Alberto Moreno Parra



Conclusiones

ML .NET ofreció la mejor curva en un 99.9% de las veces. Una razón de este resultado fue que en Math.Numerics se trabaja con una ecuación modelo (que el desarrollador escribe) y la librería ajusta los coeficientes de esa ecuación. En cambio, ML .NET con FastTree hace uso de boosting de gradiente (es una técnica de aprendizaje automático utilizada para mejorar la precisión de los modelos predictivos).

Autor: Rafael Alberto Moreno Parra